CODEALPHA ROBOTICS & AUTOMATION INTERNSHIP

TASK 3: AUTOMATION SYSTEM DESIGN

Sensor-Based Conveyor Belt Automation: circuit, flowchart and tested control logic

Meet Agrawal | B.Tech. Robotics & Artificial Intelligence | Ramdeobaba University, Nagpur | Roll No. 17 | 2026–27

Aim

To design a conveyor that stops when an object reaches a work station, holds it for a processing step (count, inspect or sort), then resumes, with a working emergency stop and fault handling. The deliverables are a block diagram, a wiring (circuit) diagram, a flowchart and state diagram, and an Arduino sketch whose logic was compiled and tested on a simulated belt.

System Overview and Components

A 12 V DC gear motor drives the belt. A photoelectric sensor (IR1) at the station detects objects; a second sensor (IR2) at the exit counts delivered items. An Arduino Uno reads the sensors and a START button, and switches the motor through a logic-level MOSFET. A two-contact emergency stop breaks motor power directly in hardware and also reports its state to the Arduino, so safety does not depend on software alone.

Part	Value / type	Function
Supply, F1	12 V DC 2 A, 2 A fuse	Powers motor; fuse protects wiring
S1 E-stop	Mushroom switch, 2 NC contacts	S1a cuts motor power; S1b tells the Arduino (open or broken wire = stop)
Buck converter	LM2596 set to 5 V	Powers Arduino and sensors from the fused 12 V
Controller	Arduino Uno (a PLC in industry)	Runs the state machine
Q1, R1, R2, D1	IRLZ44N, 220 Ω, 10 kΩ, 1N5819	Low-side motor switch; gate resistor; pull-down; flyback diode across motor
IR1, IR2	E18-D80NK, NPN normally open	Object detection and exit counting; output goes LOW on detection
S2	Push button, normally open	START / RESET (deliberate restart)
LED1/2, BZ1	Green, red (330 Ω); 5 V buzzer via BC547	Run, stop/fault and alarm indication

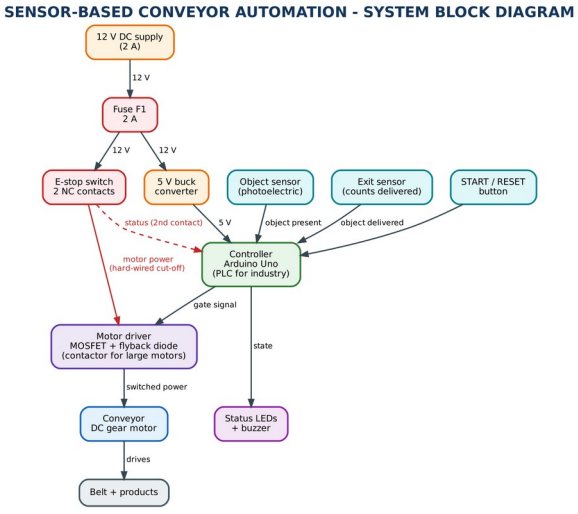


Figure 1. System block diagram.

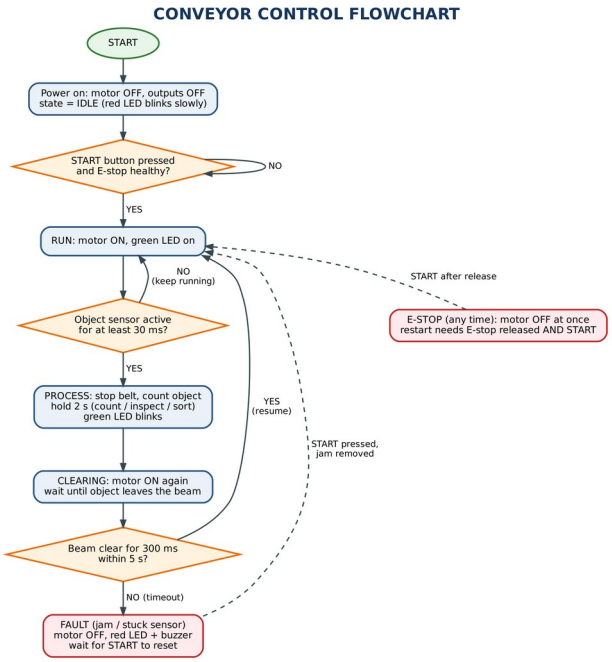


Figure 2. Control flowchart.

Wiring Diagram

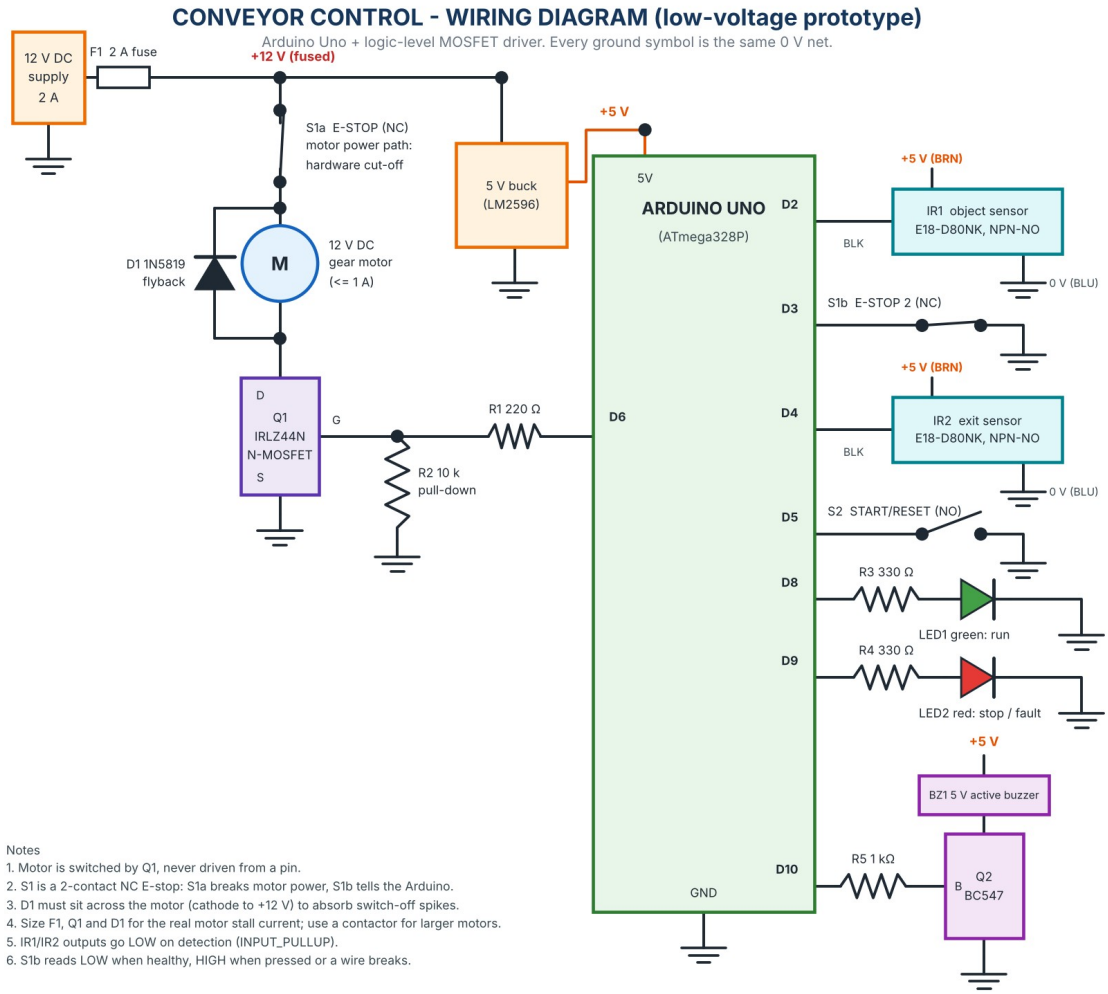


Figure 3. Wiring diagram of the prototype (pins match the sketch: D2 object sensor, D3 E-stop monitor, D4 exit sensor, D5 START, D6 motor gate, D8/D9 LEDs, D10 buzzer).

The motor is switched by Q1, never driven from a pin. D1 absorbs the voltage spike when the motor switches off. R2 keeps the gate low while the Arduino resets, so the belt cannot start by itself at power-up. Both sensors are wired to inputs with the internal pull-up, so a detection pulls the pin LOW. The E-stop monitor input reads LOW only while the contact is closed, so a pressed button and a broken cable look the same and both stop the belt.

Control Logic

State	Motor	Moves to the next state when
IDLE	off	START pressed and E-stop healthy → RUN (nothing starts at power-up)
RUN	on	object sensor active for 30 ms → PROCESS (belt stops, object counted)
PROCESS	off	2 s dwell finished → CLEARING (time for count / inspect / sort)
CLEARING	on	beam clear for 300 ms → RUN; not clear after 5 s → FAULT (jam or stuck sensor)
FAULT	off	START pressed → CLEARING if an object is still held, otherwise RUN; buzzer sounds
ESTOP	off	E-stop open in any state → ESTOP at once; restart needs E-stop released AND START

Inputs are debounced for 30 ms so electrical noise cannot trigger a detection, but an E-stop reading acts immediately. An object that is already counted when the belt is stopped by an E-stop or fault is moved out of the beam before normal detection resumes, so it is never counted twice. The full sketch is Design/Conveyor_Control/Conveyor_Control.ino, and the state diagram is in Diagrams/State_Diagram.png.

Testing

The real sketch was compiled on a PC against a small Arduino shim and run on a simulated belt (100 mm/s, 60 mm objects, sensor beam at 500–540 mm). Five scenarios gave 22 of 22 checks passed. One test initially failed and exposed a real bug (an object left in the beam after an E-stop was counted twice); it was fixed and retested.

Scenario	Result
Three objects, normal run	Belt stopped with the object 500–510 mm along (at the sensor), held 2.00 s, then resumed; 3 counted at the station and 3 at the exit
Power-up	Motor stays off in IDLE until START is pressed
Sensor noise	Fifty 10 ms glitches ignored; a sustained 60 ms signal accepted
Emergency stop	Motor off in the same 1 ms control cycle; releasing alone does not restart; START while held is ignored; works in RUN and PROCESS; no double count on restart
Jam / stuck sensor	FAULT after the 2 s dwell plus the 5 s timeout, motor off, buzzer pulsing; START resets once the jam is removed

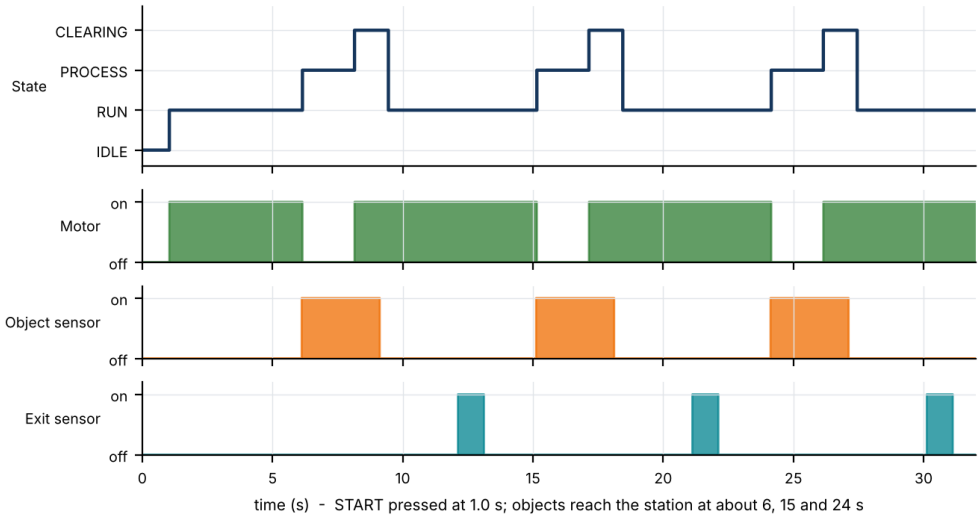


Figure 4. Simulated run of the sketch: three objects stop the belt at the station, are held for 2 s, then released; the exit sensor counts them later.

Safety and Limitations

This was tested in simulation, not on hardware, and the circuit has not been built; component values are a starting point and must be checked against the actual motor and sensors. Size the fuse, MOSFET and flyback diode for the motor's stall current and use a contactor or driver module for larger motors. The software emergency stop is a back-up: the real stop is S1a in the motor power path, and an industrial machine needs a safety-rated relay or safety PLC. Set the sensor position from the belt speed and stopping distance, guard the rollers and isolate power before maintenance. The same states map directly onto a PLC ladder program (sensor and button inputs, motor and lamp outputs, timers for dwell and jam timeout).

Conclusion

A sensor, controller and motor driver were combined into a conveyor that stops at a station, processes, resumes, and handles an emergency stop and a jam. The design is documented with a block diagram, wiring diagram, flowchart and state diagram, and its control logic passed all simulated checks, ready for a hardware build and risk assessment.